

Assignment 4: Containerized Web Application

LEMP Containers – Rahti / OpenShift Deployment and Database Integration

1. Objective of the Assignment

The aim of this assignment is to build a small web application using containers and deploy it on Rahti. The application has three main parts: a frontend, a backend REST API and a MySQL database. The frontend communicates with the backend, and the backend reads and writes data in MySQL. The project is then prepared for deployment using Kubernetes/OpenShift YAML files.

2. Project Structure in Visual Studio Code

The project was organised in a simple folder structure. The backend folder contains the Python API and its Dockerfile. The frontend folder contains the HTML page and Nginx configuration. The db/init folder contains the SQL used to initialise MySQL. The Rahti folder contains the deployment, service, route, ConfigMap, Secret and persistent-volume YAML files.

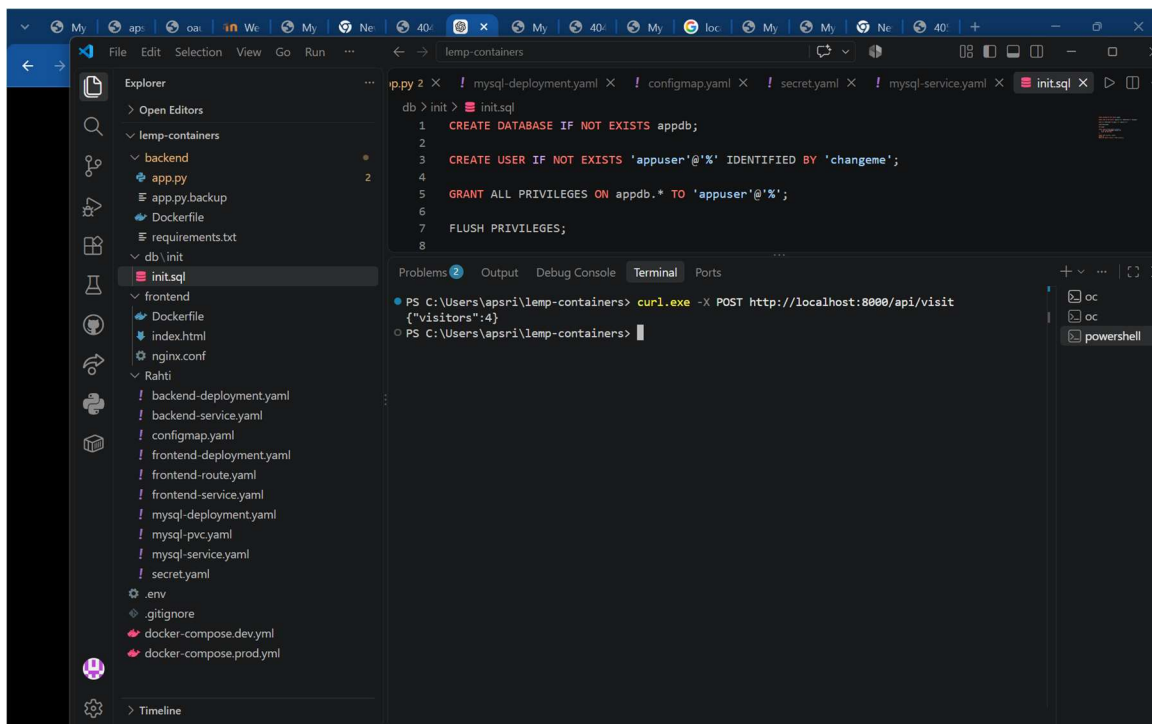


Figure 1. Visual Studio Code project structure and application preparation

This structure is useful because each part has a clear job. For example, frontend files can be changed without changing the database configuration while the YAML files decide the cloud services configurations.

2.1 Database Initialization

The init.sql file prepares the MySQL database. It creates the appdb database. The project also contains mysql-deployment.yaml, mysql-pvc.yaml and mysql-service.yaml. Together, these files describe the MySQL container, its storage and the internal service used by the backend.

3. REST API Write Operation

A REST API is used as the communication path between the frontend and backend. In this project, the endpoint `/api/visit` is used for the visitor-count write operation. A POST request tells the backend to update the visitor count in MySQL. The terminal test shows that the returned JSON value increases as requests are sent.

The figure consists of two screenshots of a Visual Studio Code editor window. The top screenshot shows the 'Terminal' tab with a series of curl commands being executed to send POST requests to the `http://localhost:8000/api/visit` endpoint. The output of each request shows the visitor count increasing from 4 to 10. The bottom screenshot shows the same terminal window with a MySQL command prompt session. The user runs `mysql> SHOW TABLES;` and `mysql> SELECT * FROM visitors;`, which returns the current state of the database, showing a single row with an id of 1 and a count of 10.

```
db > init > init.sql
1 CREATE DATABASE IF NOT EXISTS appdb;
2
3 CREATE USER IF NOT EXISTS 'appuser'@'%' IDENTIFIED BY 'changeme';
4
5 GRANT ALL PRIVILEGES ON appdb.* TO 'appuser'@'%';
6
7 FLUSH PRIVILEGES;
8

PS C:\Users\apsri\lemp-containers> curl.exe -X POST http://localhost:8000/api/visit {"visitors":4}
PS C:\Users\apsri\lemp-containers> curl.exe -X POST http://localhost:8000/api/visit {"visitors":5}
PS C:\Users\apsri\lemp-containers> curl.exe -X POST http://localhost:8000/api/visit {"visitors":6}
PS C:\Users\apsri\lemp-containers> curl.exe -X POST http://localhost:8000/api/visit {"visitors":7}
PS C:\Users\apsri\lemp-containers> curl.exe -X POST http://localhost:8000/api/visit {"visitors":8}
PS C:\Users\apsri\lemp-containers> curl.exe -X POST http://localhost:8000/api/visit {"visitors":9}
PS C:\Users\apsri\lemp-containers> curl.exe -X POST http://localhost:8000/api/visit {"visitors":10}
PS C:\Users\apsri\lemp-containers>

PS C:\Users\apsri\lemp-containers> curl.exe -X POST http://localhost:8000/api/visit {"visitors":9}
PS C:\Users\apsri\lemp-containers> curl.exe -X POST http://localhost:8000/api/visit {"visitors":10}
PS C:\Users\apsri\lemp-containers> C:\Users\apsri\Downloads\oc.exe exec -it mysql-7c776f4bdc-64521 -n aps -- bash
bash-5.1$ mysql -u appuser -p appdb
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 22
Server version: 8.4.11 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW TABLES;
+-----+
| Tables_in_appdb |
+-----+
| visitors         |
+-----+
1 row in set (0.01 sec)

mysql> SELECT * FROM visitors;
+-----+
| id | count |
+-----+
| 1  | 10    |
+-----+
1 row in set (0.00 sec)
```

Figure 2. Repeated POST requests to `/api/visit` and MySQL verification

The test proves that the API is not returning a fixed number. Each successful POST request causes the backend to perform a database update, and the next response contains the new count.

POST `/api/visit` is accepted by the backend.

The backend returns JSON containing the visitor count.

The count increases after successive write requests.

The value is stored in MySQL, so the result is connected to real database state.

3.1 Direct MySQL Verification

After the API test, the MySQL client was used to check the database directly. `SHOW TABLES;` displayed the visitors table. `SELECT * FROM visitors;` showed the stored record. The final count was 10, matching the final value returned by the API test.

4. Local Container Test

Before cloud deployment, the application can be tested locally with Docker Compose. The local setup contains frontend, backend and MySQL services. The screenshot below shows the Docker Desktop project with these three services and the application logs.

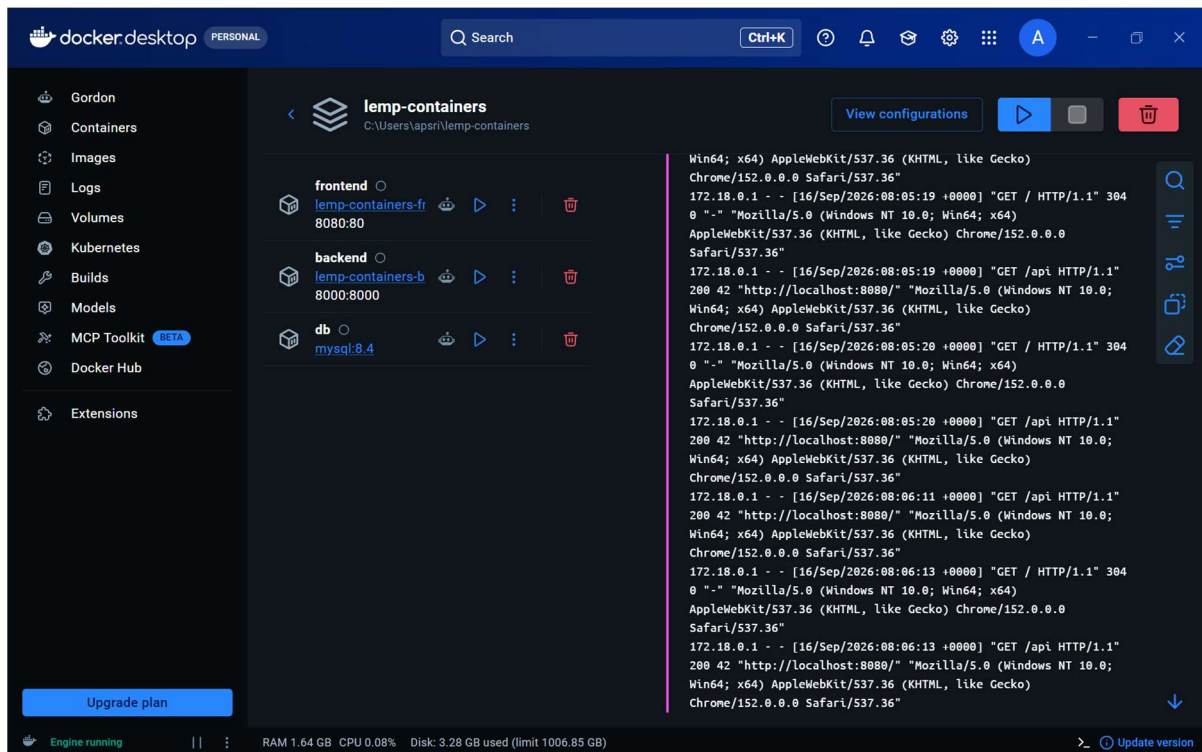


Figure 3. Docker Desktop showing the LEMP multi-container application

The frontend is exposed on port 8080 and the backend uses port 8000 in the development configuration. MySQL runs as the database service. The services communicate through the Docker network instead of exposing the database directly to the outside.

5. Deployment on Rahti / OpenShift

Rahti is the CSC container cloud environment used for this deployment. The application is placed inside the project namespace 'aps'. In Rahti, the application is represented by OpenShift/Kubernetes resources such as Deployments, Pods, Services and Routes. The Rahti web interface gives a graphical view of these resources.

5.1 Rahti Web Interface

The Rahti screenshot below shows the 'aps' project in the Workloads view. Three deployments are visible: assignment-2, backend and mysql. The right side shows the assignment-2 service and the assignment-2 route. The route provides the public URL through which the application can be reached.

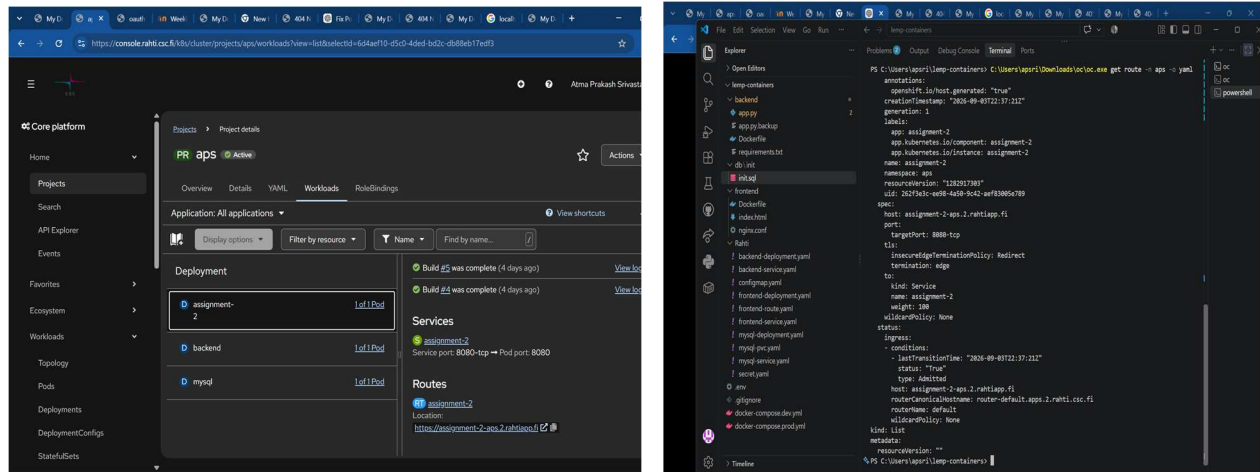


Figure 4. Rahti Web UI (left) and the corresponding terminal route output (right)

The two screenshots support each other. The Rahti UI shows the running workloads, service and route visually. The terminal output gives the exact Route object values: namespace `aps`, host `assignment-2-aps.2.rahtiapp.fi`, target port `8080-tcp`, edge TLS termination and an admitted ingress status.

Application URL: <https://assignment-2-aps.2.rahtiapp.fi>

6. Git and GitHub Repository

Git was used to keep a version-controlled copy of the project. The local project was initialised as a Git repository, the project files were added, and an initial commit was created. The local repository was then connected to the GitHub remote and pushed to the main branch.

The '.gitignore' file prevents local environment files such as `.env` from being included in the repository as the file contains passwords or other configuration values.

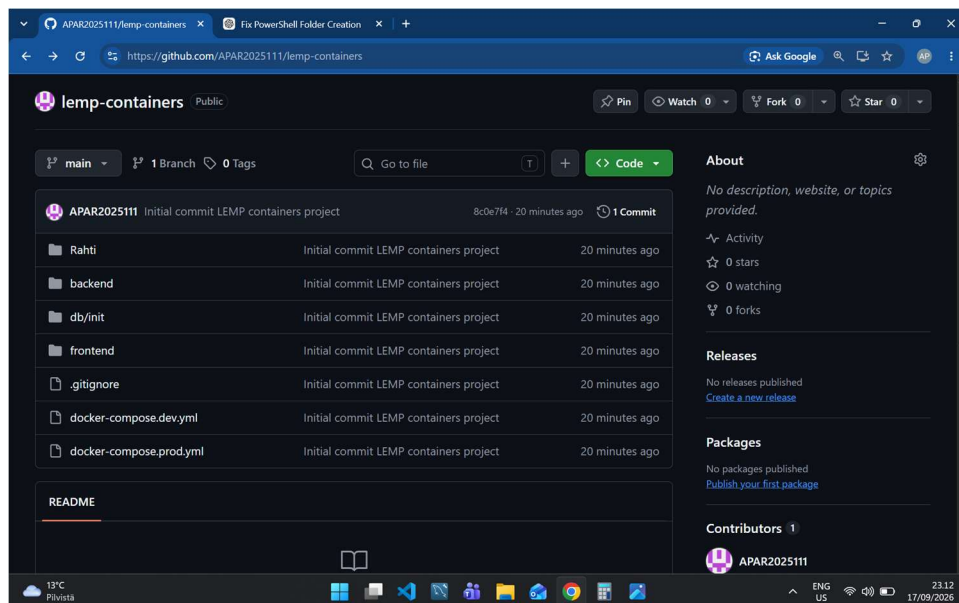


Figure 5. Public GitHub repository lemp-containers after the initial push

The GitHub page shows the repository name 'lemp-containers', the public visibility label, the main branch and the project folders/files including Rahti, backend, db/init, frontend, .gitignore and the Docker Compose files.

7. Persistence, ConfigMap and Secret

The MySQL database needs persistent storage because a database should not lose its data simply because a Pod is recreated. The project includes mysql-pvc.yaml for persistent volume configuration. The database data is stored in the visitors table, and the earlier MySQL check confirmed the stored count.

The project also contains configmap.yaml and secret.yaml. A ConfigMap is suitable for ordinary configuration values, while a Secret is intended for sensitive values such as passwords. Separating these values from the application image makes the deployment easier to manage and reduces the need to hard-code configuration.

mysql-pvc.yaml → Persistent storage for MySQL → Present in Rahti folder

configmap.yaml → Non-sensitive configuration → Present in Rahti folder

secret.yaml → Sensitive values such as passwords → Present in Rahti folder

mysql-service.yaml → Internal service for MySQL → Present in Rahti folder

8. Verification Summary

REST API write operation → POST /api/visit returned increasing JSON count → Tested

Database write → visitors table showed count = 10 → Verified

Rahti route → assignment-2 route with admitted ingress → Verified

Persistent storage → mysql-pvc.yaml present → Configured

Configuration → ConfigMap and Secret manifests present → Configured

9. Conclusion

The completed work demonstrates an end-to-end containerized web application. The frontend and backend are separated into containers, the backend communicates with MySQL, and the visitor count is updated through the REST API. The database result was checked directly in MySQL, which confirms that the write operation reached the database.

The Rahti evidence shows the application running in the 'aps' project with assignment-2, backend and mysql workloads. The Route is admitted and provides the public application URL. The project also includes persistent storage, ConfigMap and Secret resources. Finally, the project has been committed and pushed to the GitHub repository lemp-containers.

Important Links

- **Final Rahti application:** <https://assignment-2-aps.2.rahtiapp.fi>
- **GitHub repository :** <https://github.com/APAR2025111/lemp-containers>